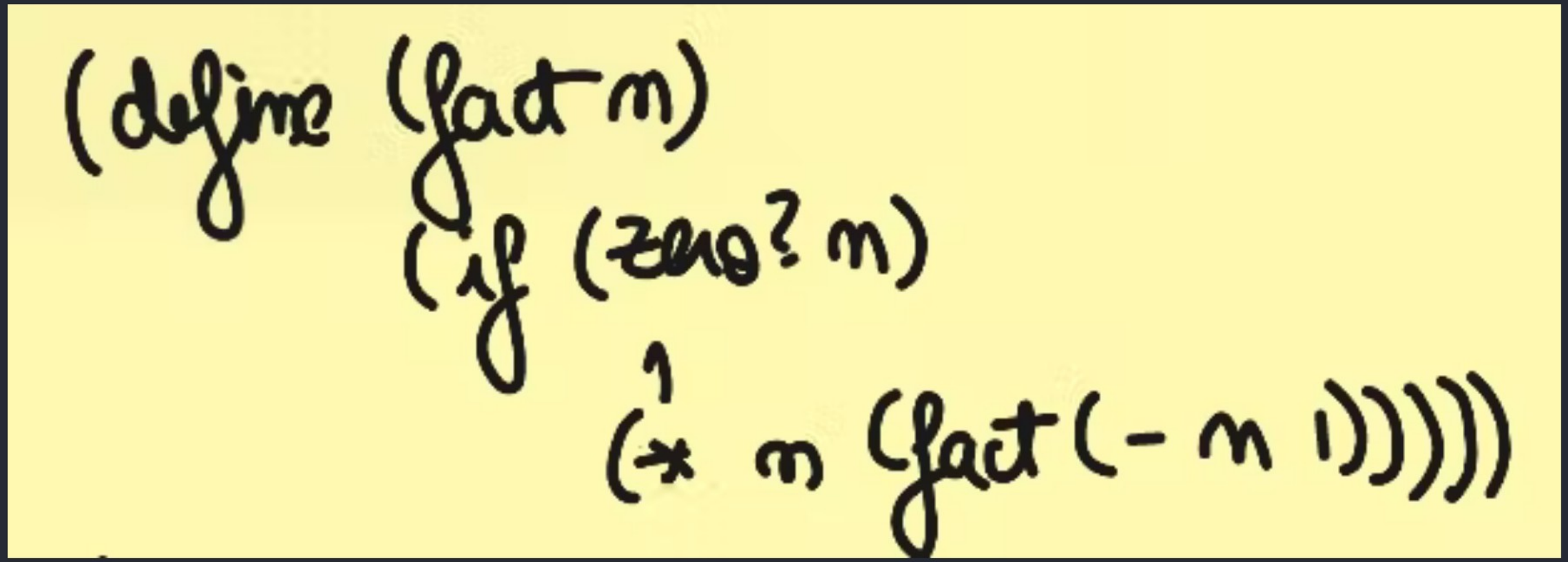
Mentimeter

Racket: Recursivitate

Mentimeter



Exemplu factorial

?

Mentimeter

Probleme?

- fiecare apel de functie **ocupa spatiu pe stiva** (spatiul ocupat O(n))
- spatiul se elibereaza atunci cand **se revine din apel**
- poate genera **stack overflow**

Mentimeter

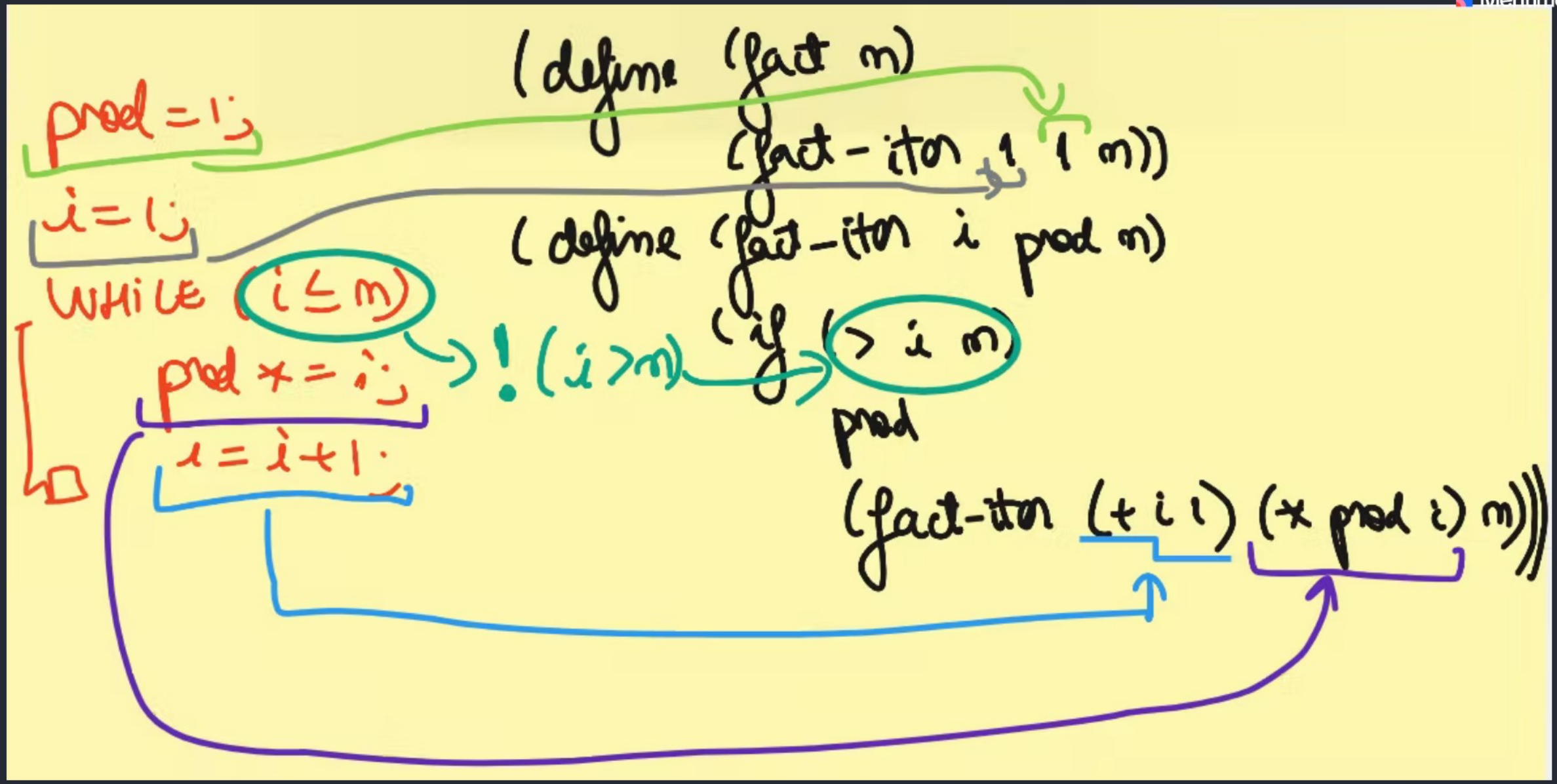
Solutie

modificam recurentele pentru a nu fi necesara retinerea de informatie pe stiva

Mentimeter

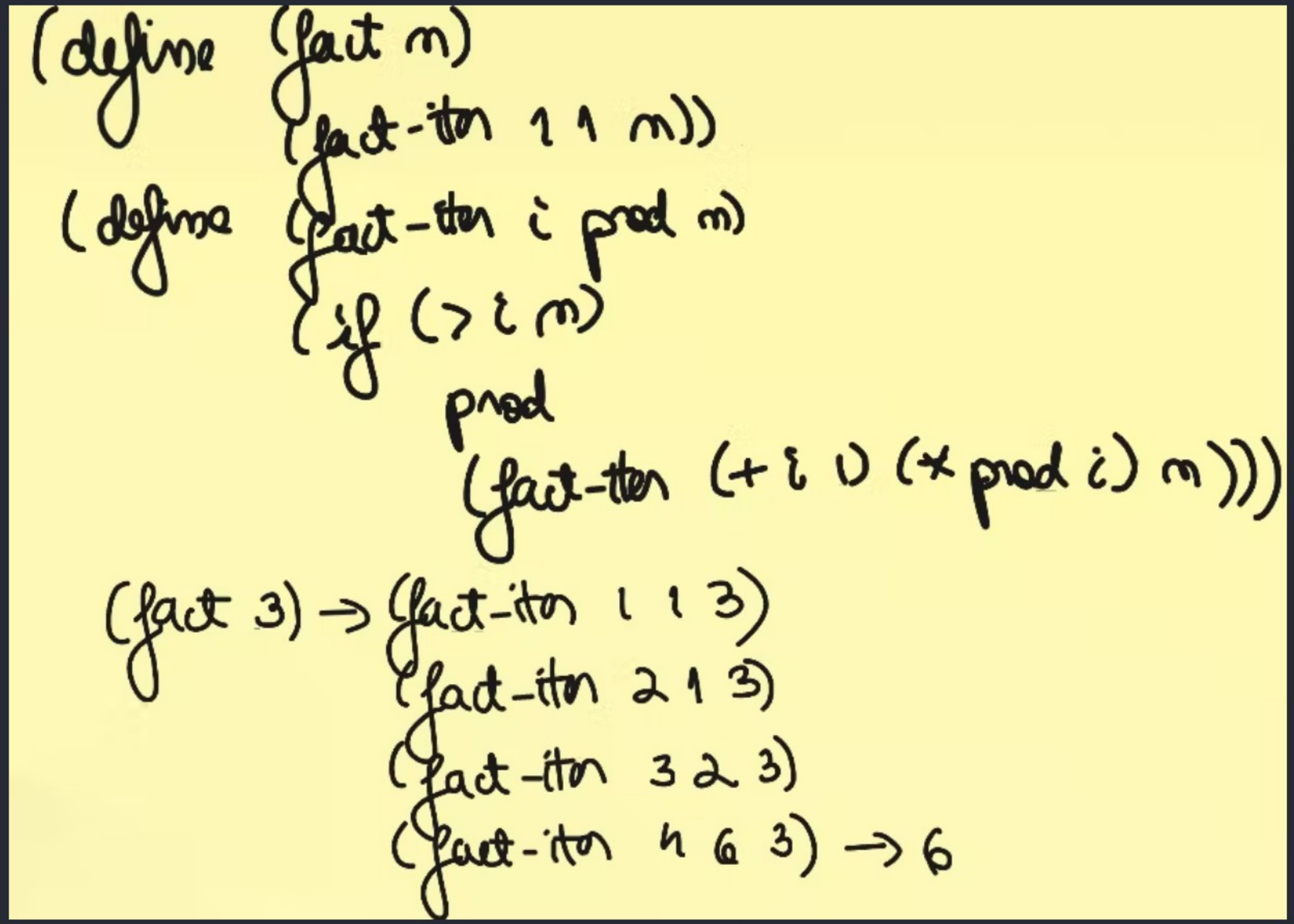
Pentru factorial

- problema: trebuie sa ajungem la 0 pentru a reveni la restul apelurilor
- solutie: incepem de la 0/1 si folosim parametri extra pentru a construi raspunsul pe parcurs



Transformare din varianta iterativa

Mentimeter



Varianta finala

Mentimeter

De ce ajuta asta?

Tail call optimization

Mentimeter

Tail call optimization

- intoarcem direct valoarea intoarsa de apelul recursiv => nu trebuie sa retinem informatia pe stiva
- putem sa inlocuim zona corespunzatoare de stiva cu cea a apelului recursiv
- spatiul ocupat O(1)

Mentimeter

Tipuri de recursivitati

- pe stiva (stack recursion)
- pe coada (tail recursion)
- arborescenta (tree recursion)

Mentimeter

Recursivitate pe stiva

- apelul recursiv face parte dintr-o expresie mai complexa
- **trebuie sa ne intoarcem din recursivitate pentru a prelucra rezultatul**

Mentimeter

Recursivitate pe coada

- putem genera rezultatul pe parcusul apelului recursiv
- **nu trebuie sa retinem informatiile pe stiva si nu trebuie sa revenim pentru a prelucra rezultatul**
- (de obicei) foloseste acumulatori pentru a construi treptat rezultatul final

Mentimeter

Recursivitatea arborescenta

- minim 2 apeluri recursive **executate neconditionat**
- **ineficienta spatial** (trebuie sa stocam stack frames pentru a reveni ulterior)
- **ineficienta temporal** (de obicei, se duplica apeluri)

Mentimeter

```
(define (fibonacci n)
  (cond
    ((zero? n) 0)
    ((equal? n 1) 1)
    (else (+ (fibonacci (- n 1)) (fibonacci (- n 2))))))
(fibonacci 10)
```

Al n-lea termen fibonacci(n >= 0) - recursivitate arborescenta

Mentimeter

Idee

- avem nevoie numai de ultimele 2 numere din secventa
- le putem construi pe parcurs folosind niste acumulatori

Mentimeter

```
(define (fibonacci n)
  (fibonacci-tail 1 0 1 n))

(define (fibonacci-tail i a b n)
  (cond
    ((zero? n) 0)
    ((equal? n 1) 1)
    ((equal? i n) b)
    (else (fibonacci-tail (+ i 1) b (+ a b) n))))

(fibonacci 0)
(fibonacci 1)
(fibonacci 2)
(fibonacci 10)
```

Transformare in recursivitate pe coada

Mentimeter

Idee 2.0

- n nu se modifica => pot sa scap de un parametru
- in loc sa maresc i pana ce ajunge egal cu n, pot sa scad n pana ce ajunge egal cu 1

Mentimeter

Problema

trebuie sa tin cont de cazul in care
n=0 din start

Mentimeter

```
(define (fibonacci n)
  (if (zero? n)
      0
      (fibonacci-tail 0 1 n)))

(define (fibonacci-tail a b n)
  (cond
    ((equal? n 1) b)
    (else (fibonacci-tail b (+ a b) (- n 1)))))

(fibonacci 0)
(fibonacci 1)
(fibonacci 3)
(fibonacci 10)
```

Recursivitate pe coada 2.0

Mentimeter

Exemplu 1

Sa se intoarca lista tuturor numerelor naturale de la 1 la n

Mentimeter

```
(define (sequence-stack n)
  (if (zero? n)
      '()
      (append (sequence-stack (- n 1)) (list n))))
(sequence-stack 10)
```

Recursivitate pe stiva

Mentimeter

```
(define (sequence-tail n)
  (sequence-tail-helper n '()))

(define (sequence-tail-helper n l)
  (if (zero? n)
      l
      (sequence-tail-helper (- n 1) (cons n l))))

(sequence-tail 10)
```

Recursivitate pe coada

Mentimeter

Exemplu 2

Sa se concateneze 2 liste A, B

Mentimeter

2 variante de recursivitate

- dupa elementele lui A vs dupa elementele lui B
- mai usor si eficient este sa facem axiomele dupa A

Mentimeter

```
(define (append-stack A B)
  (if (null? A)
      B
      (cons (car A) (append-stack (cdr A) B))))

(append-stack '(1 2) '(3 4))
```

Recursivitate pe stiva

Mentimeter

```
(define (append-tail A B)
  (append-tail-helper A B))

(define (append-tail-helper A result)
  (if (null? A)
      result
      (append-tail-helper (cdr A) (cons (car A) result))))

(append-tail '(1 2) '(3 4))
```

Recursivitate pe coada

Mentimeter

Ce e gresit?

- elementele din A sunt adaugate in ordine inversa
- ex: A='(1 2), B='(3 4), mai intai se adauga 1, dupa 2=>result='(2 1 3 4)
- practic (append-tail-helper A B) calculeaza acum (append (reverse A) B)

Mentimeter

Solutie 0

- inversam A inainte sa apelam append-tail-helper
- (append-tail-helper A' B) = (append (reverse A') B) = (append A B)
- am notat cu A' (reverse A) => (reverse A') = (reverse (reverse A)) = A

Mentimeter

```
(define (append-tail A B)
  (append-tail-helper (reverse A) B))

(define (append-tail-helper A result)
  (if (null? A)
      result
      (append-tail-helper (cdr A) (cons (car A) result))))

(append-tail '(1 2) '(3 4))
```

Solutie 0

Mentimeter

Solutie 1 (cheatsheet)

- inversam ordinea parametrilor
- daca (append-tail-helper A B) = (append (reverse A) B)
- atunci (append-tail-helper B (reverse A)) = (append (reverse B) (reverse A))
- ca sa obtinem (append A B) mai inversam o data rezultatul final

Mentimeter

```
(define (append-tail1 A B)
  (append-tail1-helper B (reverse A)))

(define (append-tail1-helper B Result)
  (if (null? B)
      (reverse Result)
      (append-tail1-helper (cdr B) (cons (car B) Result))))
(append-tail1 '(1 2) '(3 4))
```

Solutie 1